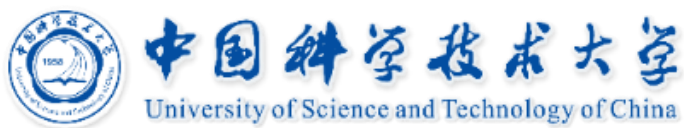


人工智能+数学物理 II

授课人：康奇宇



人工智能辅助数学研究

定理证明 · 规律发现 · 神经离散优化与复杂度分析

01

总体框架：从数学实验到机器可验证证明

AI 的三个作用方向与统一的约束搜索抽象

02

辅助定理证明与形式化验证

证明对象、Curry-Howard 对应、证明搜索与 SAT / SMT

03

数据驱动的规律发现与猜想提出

符号回归、递推与生成函数、图不变量、动力系统发现

04

神经离散优化：让网络求解组合问题

五种范式：自回归构造、连续松弛、能量模型、学习迭代、增强求解器

05

复杂度优化、自动发现与统一视角

复杂度的四个层次、自动反例搜索、理想的研究闭环

人工智能辅助数学研究的三个方向

1 辅助定理证明与形式化验证

在巨大的证明空间中搜索候选证明，并由形式系统检查每一步推理的正确性。

$$\Gamma \vdash \pi : \varphi$$

2 数据驱动规律发现

从数值数据、离散对象或计算实验中发现潜在结构，提出可检验的数学猜想。

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N \longrightarrow f^*$$

3 算法复杂度优化与离散数学

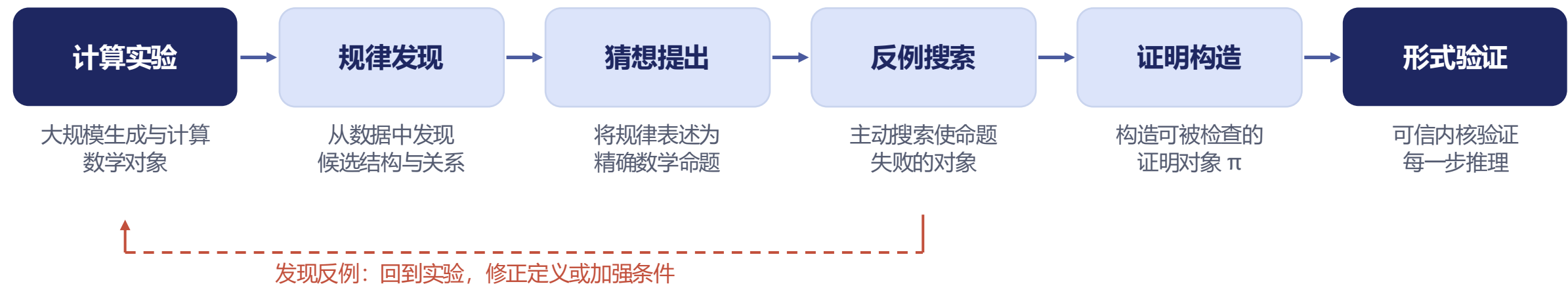
改进组合搜索、离散优化和算法设计中的关键决策，提高典型实例上的效率。

$$\min_{x \in \mathcal{X}} c(x)$$

三个方向彼此联系：发现告诉我们“证什么”，证明告诉我们“为什么成立”，算法决定“能否搜索得动”。

本讲主线 AI 最适合扩展数学家的**搜索半径**：候选公式搜索、引理检索、证明步骤生成与离散结构枚举；选择定义、发现不变量、解释定理为什么成立，仍然由数学家完成。

一个完整的研究闭环



前半程：发现（AI 占优）

计算实验、规律发现与反例搜索都是**大规模搜索与枚举**，正是机器擅长的部分：算得多、试得快、不知疲倦。

后半程：确证（人机协作）

猜想表述、证明构造需要数学洞察；**形式验证**则把“看起来对”升级为“机器可检查地对”。

闭环要点 每个箭头都可以由 AI 加速，但任何结论只有走完“反例搜索 + 形式验证”才算可靠。

核心观点：AI 扩展数学家的搜索半径

AI 擅长：搜索与枚举

- **候选公式搜索**：在表达式空间中大规模尝试
- **引理检索**：从庞大定理库中找到可用结论
- **证明步骤生成**：为当前目标建议下一个 tactic
- **离散结构枚举**：系统生成图、程序与组合对象

数学家不可替代：选择与解释

- **选择正确的定义**：什么才是问题的本质对象
- **发现不变量**：哪些量在变换下保持不变
- **提炼结构性引理**：哪个中间命题有概念意义
- **解释定理为什么成立**：从“正确”到“理解”

比喻：AI 像高倍望远镜，看得更远、扫得更宽；往哪里看、看见的东西意味着什么，仍由数学家决定。

分工原则 机器扩大**搜索半径**（搜公式、搜引理、搜步骤、搜结构），人类负责**方向与意义**（定义、不变量、结构性引理、概念性解释）。两者相乘，而不是相互替代。

统一抽象：候选空间中的约束搜索

许多数学研究任务都可以写成同一种形式：在巨大候选空间中寻找满足约束、并尽可能优化目标的对象。

$$s \in \mathcal{S}, \quad \mathcal{C}(s) = \text{True}$$

$$s^* = \arg \min_{s \in \mathcal{S}} J(s)$$

候选对象 s 可以是：

证明

公式

图

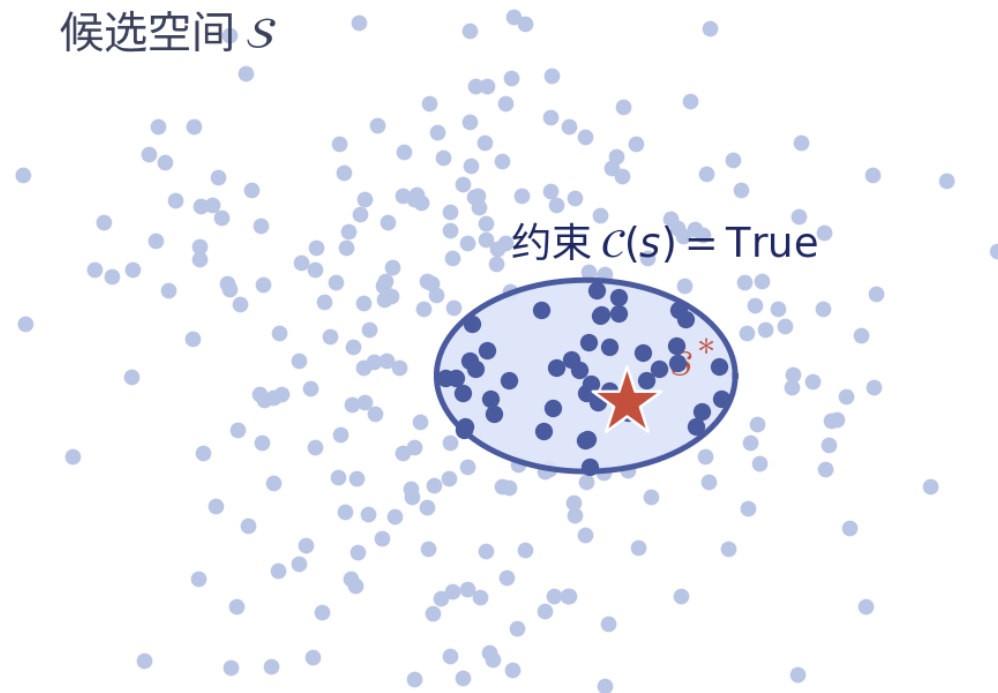
算法

程序

离散结构

- 约束 $\mathcal{C}(s)$ ：证明合法、公式拟合数据、结构满足性质
- 目标 $J(s)$ ：证明长度、表达式复杂度、路径代价、运行时间

候选空间 $\mathcal{S}$



约束 $\mathcal{C}(s) = \text{True}$

在候选云 $\mathcal{S}$ 中：先满足约束（椭圆内），再优化目标（★）

统一视角 定理证明、规律发现、算法优化，都是这同一个“约束搜索”模板的不同实例。

PART 2

辅助定理证明与形式化验证

AI 作为证明搜索引擎：让每一步推理可被机器检查

- 证明对象 π 与 Curry-Howard 对应
- 证明状态、tactic 与模型引导搜索
- 生成器-验证器架构与 SAT / SMT / 计算机代数
- 形式化的真正难点：隐含假设显式化

自然语言证明 vs 形式化证明

设 Γ 为公理、定义、变量类型和已知定理构成的上下文， φ 为待证明命题。证明问题写为：

$$\Gamma \vdash \varphi$$

自然语言证明：写给人看

- 背景知识与常识默认省略
- 类型信息隐含在记号习惯中
- 正则性条件常被一句“显然”带过
- **优点：**简洁、可读、传递直觉

形式化证明：写给机器查

- 所有对象、条件、推理规则显式编码
- 每个符号都有确定类型
- 每一步都能被可信内核机械检查
- **优点：**正确性可被独立验证

关键差别 自然语言证明的“说服力”依赖读者补全细节；形式化证明把全部细节交给机器——这正是 AI 可以介入搜索的舞台。

证明对象：让“证明”成为可检查的数据

我们不仅要知道命题成立，更希望构造一个明确的证明对象 π ，把每一步推理记录下来：

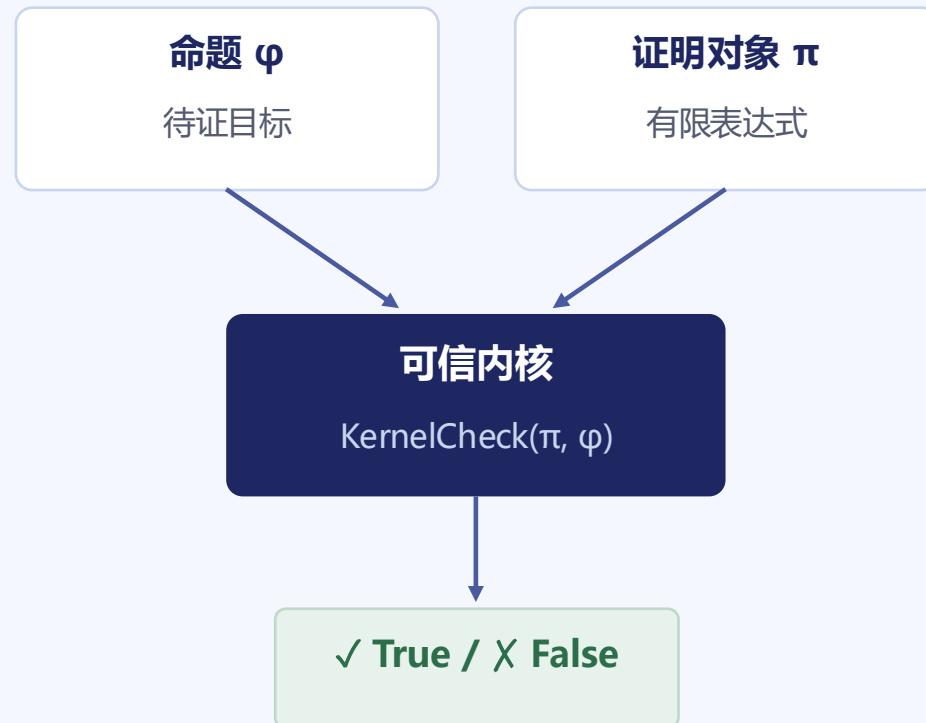
$$\Gamma \vdash \varphi \implies \Gamma \vdash \pi : \varphi$$

定义：证明对象

证明对象 π 是一个可由形式系统检查的有限表达式：它不仅说明命题 φ 成立，还记录了每一步推理所使用的规则和依赖。

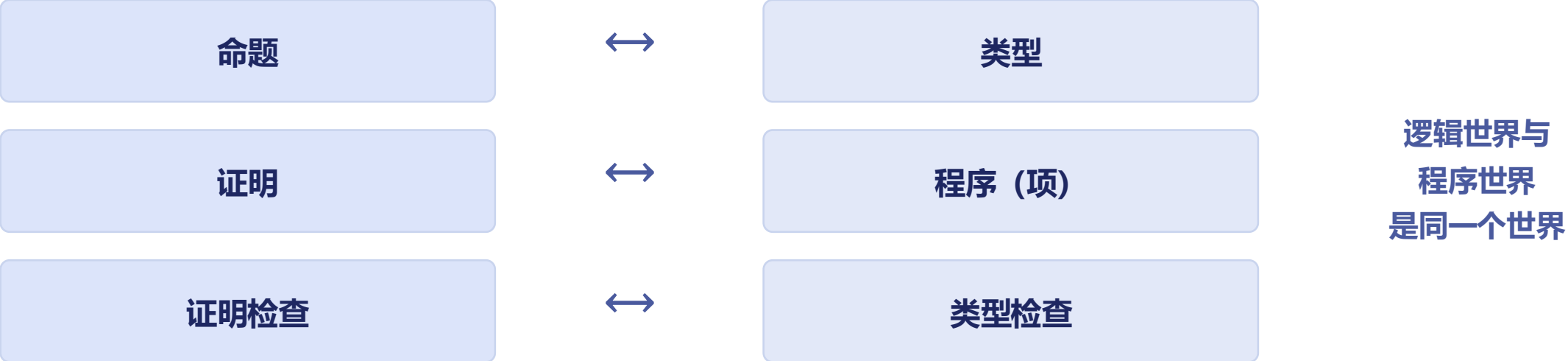
- 证明从“论述”变成一份结构化数据
- 因此可以被搜索、被存储、被自动检查

形式系统如何看待一个证明

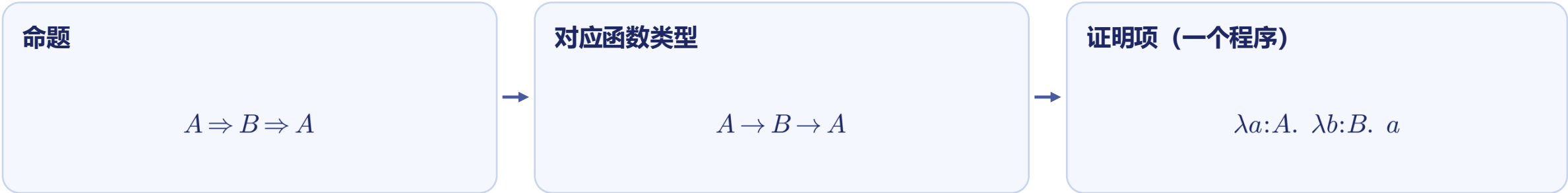


检查是机械的：逐条核对每步规则与依赖

Curry-Howard 对应：命题即类型，证明即程序



例：命题与它的“程序证明”



验证器做什么 检查“该项是否确实具有目标类型”——证明检查由此变成程序的类型检查。

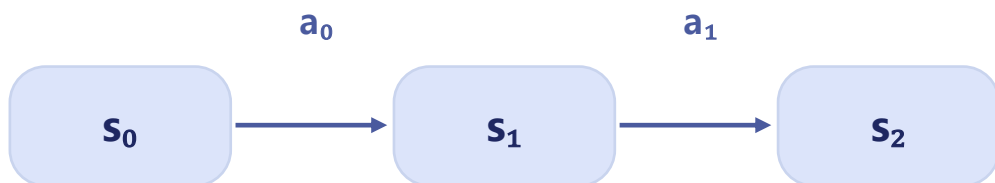
证明搜索：状态、目标与 tactic

一个尚未完成的证明 = 一组待解决目标（每个目标带自己的上下文）：

$$s_t = \{ \Gamma_1 \vdash G_1, \dots, \Gamma_k \vdash G_k \}$$

第 t 步选择一个动作 (tactic) a_t ，状态更新：

$$s_{t+1} = T(s_t, a_t)$$



terminate: $s_T = \emptyset$ (no goals left)

每一步可选的动作 (tactic)

- 使用某个已知引理
- 展开定义或重写等式
- 选择归纳变量
- 将目标拆分成若干子目标
- 调用 **线性算术 / 环 / 域 / SAT / SMT** 求解器
- 构造中间命题或辅助引理

一句话 证明 = 在“目标集合”状态机上，找到一条把目标清空的动作序列。

用模型为证明搜索“指路”

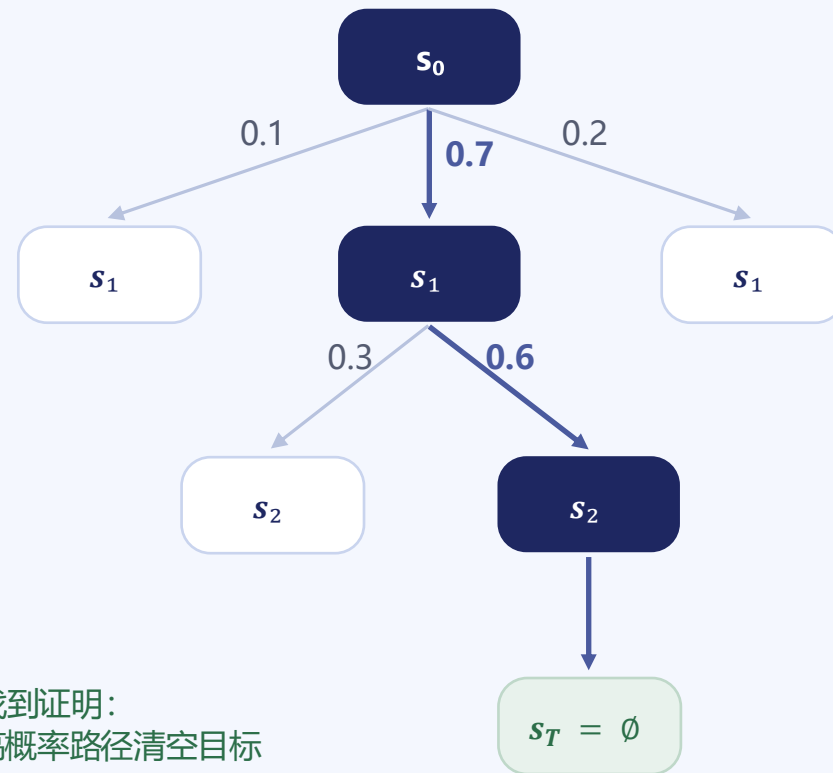
若用参数为 θ 的模型为每个状态预测动作概率，证明搜索就成为最大化序列概率：

$$\pi^* = \arg \max_{a_1, \dots, a_T} \prod_{t=1}^T p_\theta(a_t | s_t)$$

subject to: $s_T = \emptyset$

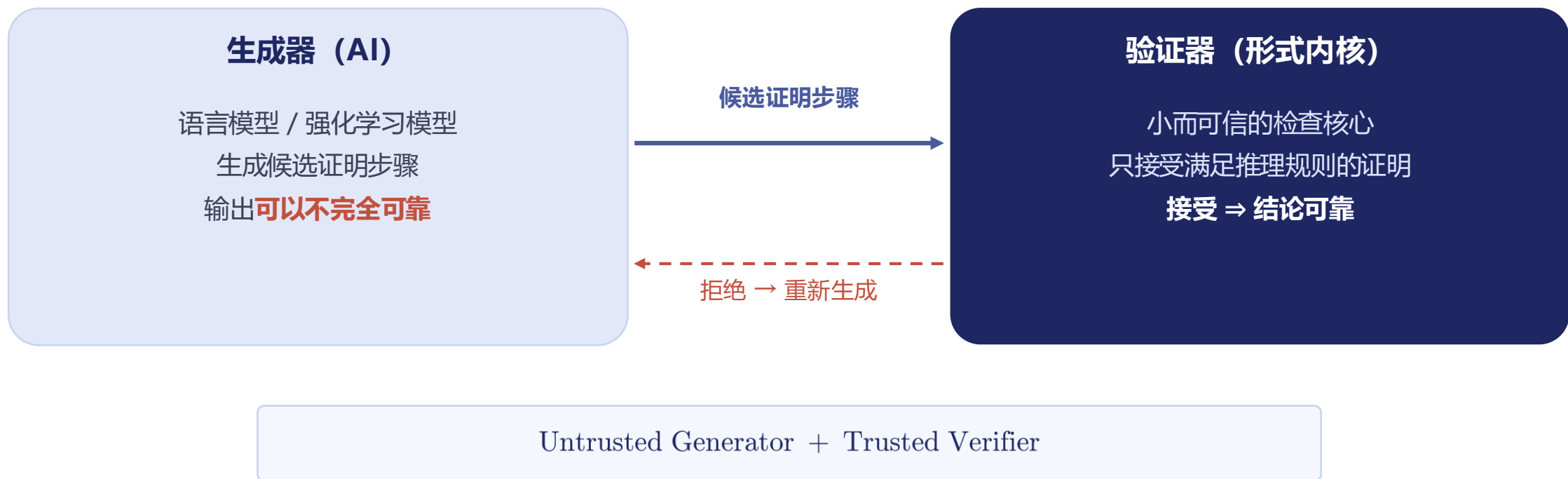
- 模型给每个分支打分，把指数大的搜索树变“有方向”
- 高概率分支优先展开，低概率分支延后或剪掉
- 搜索器可以是最佳优先、beam、MCTS 等

由 p_θ 引导的证明搜索树



角色定位 模型不保证每一步都对——它只负责让“值得先试”的分支排在前面。

总体结构：不完全可信的生成器 + 高度可信的验证器



- **正确性押在验证器上**：哪怕生成器“胡说”，错误证明也无法通过内核
- **生成器可以大胆探索**：试错成本低，因为有验证器兜底

贯穿全讲 这一“生成器-验证器”结构将在猜想检验、程序搜索与神经增强求解器中反复出现。

示例：有限求和公式的归纳证明

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

自然的证明策略：对 n 做数学归纳

基例 ($n = 0$)

$$\begin{aligned}\sum_{k=1}^0 k &= 0 \\ &= \frac{0(0+1)}{2} \quad \checkmark\end{aligned}$$

归纳步骤 (假设 n 成立, 证 $n+1$)

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \sum_{k=1}^n k + (n+1) \\ &= \frac{n(n+1)}{2} + (n+1) \quad (\text{induction hypothesis}) \\ &= \frac{(n+1)(n+2)}{2} \quad \checkmark\end{aligned}$$

例：AI 可以自动完成哪些部分

自动建议归纳变量 n 、调用归纳假设、选择适当的重写规则，并把最后一步交给**环等式求解器**；形式系统逐步验证每个等号与推理规则是否合法。

两类自动推理引擎：SAT 与 SMT

SAT: 命题逻辑可满足性

给定布尔公式，问是否存在赋值使其为真：

$$\exists x \in \{0, 1\}^n : F(x_1, \dots, x_n) = 1$$

NP 完全，但现代求解器在大量实际实例上极其高效

SMT: 带理论的可满足性

在命题逻辑之上加入代数、数组、位向量等理论，可直接处理算术约束系统。

$$x + y \leq 1, \quad x \geq 1, \quad y \geq 1$$

这个系统可满足吗？

SMT 求解器内部的一次推理：

$$x \geq 1, \quad y \geq 1$$



$$x + y \geq 2$$



contradicts $x + y \leq 1$

UNSAT
不可满足

在证明中的角色 把目标中的“逻辑/算术琐事”外包给 SAT / SMT，让搜索专注真正的结构性步骤。

注意： SAT 返回的赋值、SMT 给出的不可满足证书都可以被独立检查——它们天然属于“生成器 + 验证器”范式。

计算机代数，以及形式化的真正难点

计算机代数：恒等式与消元

验证多项式恒等式可化为验证差为零：

$$p(x_1, \dots, x_n) = q(x_1, \dots, x_n) \iff p - q \equiv 0$$

- 所有单项式系数为零 $\Rightarrow$ 恒等式成立
- **Gröbner 基、结果式、量词消去**处理更复杂的代数约束

真正的瓶颈：把数学“翻译”成形式语言

自然语言里一句 **“设函数 f 足够光滑”**，

在形式系统中必须明确为以下之一：

$$f \in C^1(\Omega) \quad f \in C^2(\Omega) \quad f \in W^{1,p}(\Omega)$$

并说明定义域、边界条件与所使用的积分理论。

经验：局部代数运算（化简、重写、解方程）机器很强；难的是没有被写出来的隐含假设——它们必须被一一显式化。

警示：形式化 $\neq$ 简单符号化

把一段自然语言证明机械地改写成 LaTeX，并不构成形式化证明。真正的形式化要求：所有符号具有确定类型、所有隐含条件显式化、每个推理步骤都能被可信内核验证。

PART 3

数据驱动规律发现与猜想提出

从计算实验与有限数据中，搜索可解释、可检验的数学结构

- 符号回归与最小描述长度 (MDL)
- 序列递推、生成函数与图不变量关系
- SINDy 动力系统发现与守恒律的零空间法
- 猜想检验、反例搜索与浮点警示

从有限数据到候选数学结构

这一方向的目标不是立即证明定理，而是从大量计算结果中发现潜在关系。典型流程：



出发点：一份有限观测数据

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

- 与普通函数逼近不同：要求输出**可解释结构**
- 黑箱拟合器预测得再准，也不构成“数学规律”

“可解释结构”指什么？

- 闭式公式：如 $y = x^2 + \sin x$
- 递推关系： a_n 由前几项线性决定
- 守恒律：某个泛函沿时间不变
- 不变量关系：图的参数之间的不等式

本章问题 如何在“能拟合数据”的无数候选中，搜索出简洁、可检验、可证明的那一个？

符号回归：搜索可解释的公式

在候选表达式空间 $\mathcal{H}$ 中寻找既拟合数据、又保持简洁的函数：

$$f^* = \arg \min_{f \in \mathcal{H}} \left[\sum_{i=1}^N |f(x_i) - y_i|^2 + \lambda C(f) \right]$$

- $C(f)$: 表达式树节点数、运算符数量或描述长度
- λ : 在“拟合好”与“写得短”之间权衡

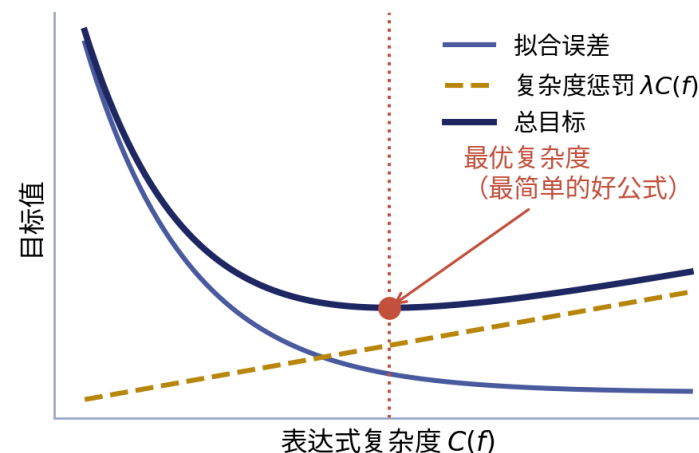
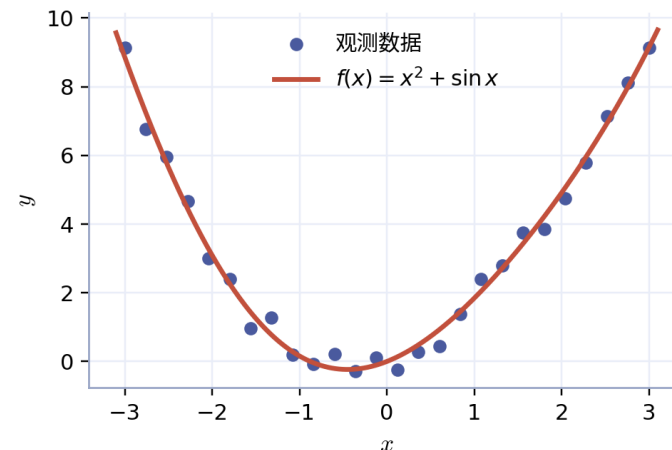
定义：奥卡姆原则与最小描述长度 (MDL)

当多个公式都能拟合数据时，优先选复杂度更低者：

好的数学规律 $\approx$ 能够解释数据的最简单表达式

正则项 $\lambda C(f)$ 正是这一原则的数学编码。

例：数据来自 $y = x^2 + \sin x$ 时，目标是直接发现这个表达式，而非一个无法解释的黑箱近似器。



序列规律：寻找低阶线性递推

给定有限序列 $a_0, a_1, \dots, a_N$ ，尝试以下假设：

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_r a_{n-r}$$

把每个 n 的等式排成线性方程组：未知系数 $c = (c_1, \dots, c_r)^T$ 由 **Hankel 型线性系统** 确定。

- **Hankel 矩阵低秩** $\Rightarrow$ 序列很可能满足低阶递推
- 解出 c 后，用更多项验证递推是否持续成立
- 系数应做有理重构，排除浮点伪规律

Hankel 矩阵：反对角线元素相同

a_2	a_1	a_0
a_3	a_2	a_1
a_4	a_3	a_2
a_5	a_4	a_3
a_6	a_5	a_4
a_7	a_6	a_5

 $\times c = [a_r, \dots, a_N]^T$

行 = 不同 n 处的递推等式；解线性系统即得系数 c

为什么重要 递推关系是序列的“压缩表示”：找到它，就把无穷多项装进了 r 个系数里。

下一步：低阶递推还能进一步导出闭式表达式与渐近增长率——这正是生成函数的工作。

生成函数：从递推到闭式与渐近

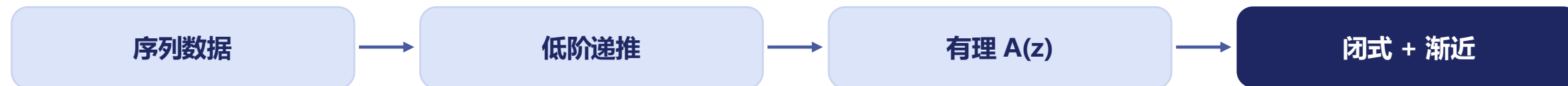
把整个序列打包成一个形式幂级数：

$$A(z) = \sum_{n=0}^{\infty} a_n z^n$$

常系数线性递推 $\iff$ 有理生成函数：

$$A(z) = \frac{P(z)}{Q(z)}$$

- 部分分式分解 $\Rightarrow$ **闭式表达式**
- $Q(z)$ 的最小模根 $\Rightarrow$ **渐近增长率**
- 从“数据 $\rightarrow$ 递推 $\rightarrow$ 生成函数”一路抵达精确结构



意义：发现递推不只是“预测下一项”，而是可能一步步抵达整个序列的精确数学结构。

例：斐波那契数列

$$a_n = a_{n-1} + a_{n-2}, \quad a_0 = 0, \quad a_1 = 1$$

$$A(z) = \frac{z}{1 - z - z^2}$$

$$a_n \sim \frac{\phi^n}{\sqrt{5}}, \quad \phi = \frac{1 + \sqrt{5}}{2}$$

两项递推 $\rightarrow$ 一个有理函数 $\rightarrow$ 黄金比例增长率

图论：在不变量之间搜寻关系

对每个图 G ，计算一组数值不变量，拼成特征向量：

$$\Phi(G) = (|V|, |E|, \chi(G), \alpha(G), \omega(G), \Delta(G), \lambda_1(G), \dots)$$

AI 在大量图样本上寻找候选关系：

$$F(\Phi(G)) \geq 0 \quad \text{or} \quad F(\Phi(G)) = 0$$

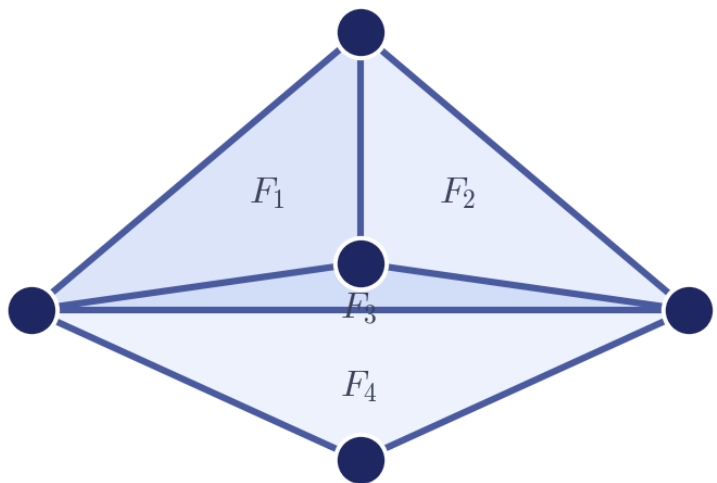
- **不等式型**：极值图论猜想（某不变量的上界 / 下界）
- **等式型**：精确恒等关系，往往暗示深层结构
- 在“随机图 + 极端图”混合样本上检验，避免偏置

常用图不变量速览

$ V , E $	顶点数、边数
$\chi(G)$	色数
$\alpha(G)$	最大独立集
$\omega(G)$	最大团
$\Delta(G)$	最大度
$\lambda_1(G)$	邻接谱半径

思路 把“看图”变成“看向量”：图的几何被压缩成 $\Phi(G)$ ，关系搜索就成了数据问题。

实例：平面图边数上界的发现与证明



$$|V| = 5, |E| = 8, |F| = 5 \text{ (含外部面)} \Rightarrow 5 - 8 + 5 = 2$$

数据先行：对大量简单连通平面图计算 $|V|$ 、 $|E|$ ，可观察到一条整齐的边界线。

观察到的规律 ($|V| \geq 3$)：

$$|E| \leq 3|V| - 6$$

真正的解释来自结构性证明：

$$\text{Euler: } |V| - |E| + |F| = 2$$

$$\text{each face } \geq 3 \text{ edges: } 3|F| \leq 2|E|$$

$$2 = |V| - |E| + |F| \leq |V| - |E| + \frac{2|E|}{3} = |V| - \frac{|E|}{3}$$

$$\Rightarrow |E| \leq 3|V| - 6 \quad \blacksquare$$

这个例子说明 数据可以帮助发现结论，但解释结论仍然依赖结构性证明——Euler 公式 + 计数论证，缺一不可。

动力系统发现：用稀疏回归找微分方程

设未知系统满足 $\dot{x}(t) = f(x(t))$

从观测轨迹构造候选函数库（字典）：

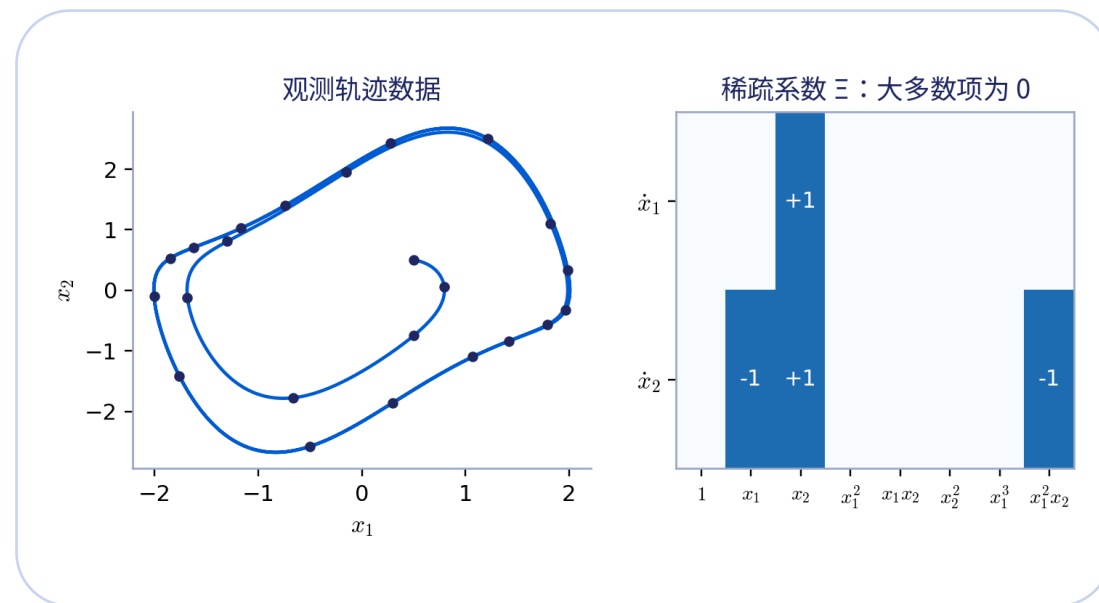
$$\Theta(x) = [1, x_1, \dots, x_d, x_1^2, x_1x_2, \dots, \sin x_1, \dots]$$

假设动力学在字典中是稀疏组合：

$$\dot{X} = \Theta(X)\Xi$$

ℓ_0 难解 $\rightarrow$ 用 ℓ_1 或逐步阈值 (SINDy)：

$$\Xi^* = \arg \min_{\Xi} [\|\dot{X} - \Theta(X)\Xi\|_F^2 + \lambda \|\Xi\|_1]$$



Van der Pol 振子：轨迹数据 $\rightarrow$ 稀疏系数矩阵 Ξ

- 非零项直接给出方程的各项与系数
- **稀疏性 = 可解释性**：写得出来的方程

目标定位 不是仅仅预测下一时刻状态，而是恢复可解释的动力学结构本身。

守恒律发现：寻找算子的零空间

对偏微分方程 $u_t = \mathcal{N}[u]$

寻找沿时间保持不变的泛函：

$$I[u] = \int_{\Omega} F(u, \nabla u, \nabla^2 u, \dots) dx, \quad \frac{d}{dt} I[u(t)] = 0$$

把候选密度展开在一组基上：

$$F = \sum_{j=1}^m c_j \phi_j \implies A c \approx 0$$

守恒律发现 = 寻找离散微分算子 **A** 的近似**零空间**：满足 $Ac \approx 0$ 的每个方向 c 都对应一条候选守恒律。

流程：从数据到守恒律

观测场数据 $u(x,t)$



候选密度库 $\{\phi_j\}$



组装线性算子 **A**



求近似零空间 **c**



候选守恒律 $I[u]$

一脉相承 符号回归找“公式”、SINDy找“方程”、零空间法找“守恒律”——同一思想：在可解释的候选基上做（稀疏）线性代数。

猜想必须经过反例搜索与证明

有限样本上的成立，推不出全称命题——这是数据驱动发现的根本限制：

$$P(x_i) = \text{True}, \quad i = 1, \dots, N \quad / \Rightarrow \quad \forall x: P(x) = \text{True}$$



可靠的猜想生成必须包含中间这步：先努力“证伪”，再尝试“证明”

警示：浮点近似的风险

若计算发现 $a \approx b$ ，还不能直接猜测 $a = b$ 。应进一步使用：

高精度计算 · 区间算术 · 有理重构 · 整数关系算法（如 PSLQ） · 符号化简 —— 排除数值误差造成的伪规律。

与方向一汇合 猜想一旦在反例搜索中幸存，就交给“证明搜索 + 形式验证”——两个方向在此闭合成环。

PART 4

神经离散优化

让神经网络参与组合优化：在哪个环节注入可学习结构？

- 组合优化的统一形式与“不可微”困难
- 范式一：自回归构造 + 掩码 + 策略梯度
- 范式二/三/四：连续松弛、能量模型、学习迭代
- 范式五：神经网络增强精确求解器（MIP / SAT）
- 评价四性质与统一数学框架

组合优化的统一形式

许多离散数学问题可以写成同一个模板：

$$\min_{x \in \mathcal{X}(I)} c(x)$$

- I : 问题实例 (图、距离矩阵、约束系统...)
- $\mathcal{X}(I)$: 随实例变化的离散可行域
- $c(x)$: 路径长度、冲突数、成本等目标

两类典型可行域：

$$\mathcal{X}(I) \subseteq \{0, 1\}^n \quad (\text{binary})$$

$$\mathcal{X}(I) = S_n \quad (\text{permutations})$$

典型问题一览 (多为 NP-hard)

旅行商 TSP

最短闭合回路

最大割 Max-Cut

切断最多边权

图着色

相邻顶点异色

SAT

布尔可满足性

整数规划 MIP

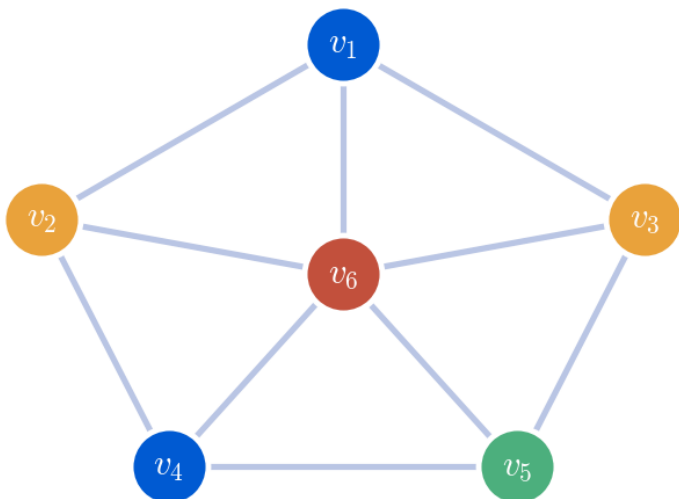
线性约束 + 整数

调度

次序与资源分配

本章问题 神经网络如何在这一离散世界中工作？它到底“求解”了什么？

实例：K-图着色的 0-1 整数编码



相邻顶点颜色不同 ($K = 4$)

一个组合问题，被完整翻译成代数约束

引入二元变量：顶点 v 是否使用颜色 c

$$x_{v,c} \in \{0, 1\}$$

约束一： 每个顶点恰好选择一种颜色

$$\sum_{c=1}^K x_{v,c} = 1 \quad \forall v \in V$$

约束二： 相邻顶点不能同色

$$x_{u,c} + x_{v,c} \leq 1 \quad \forall (u, v) \in E, \quad c = 1, \dots, K$$

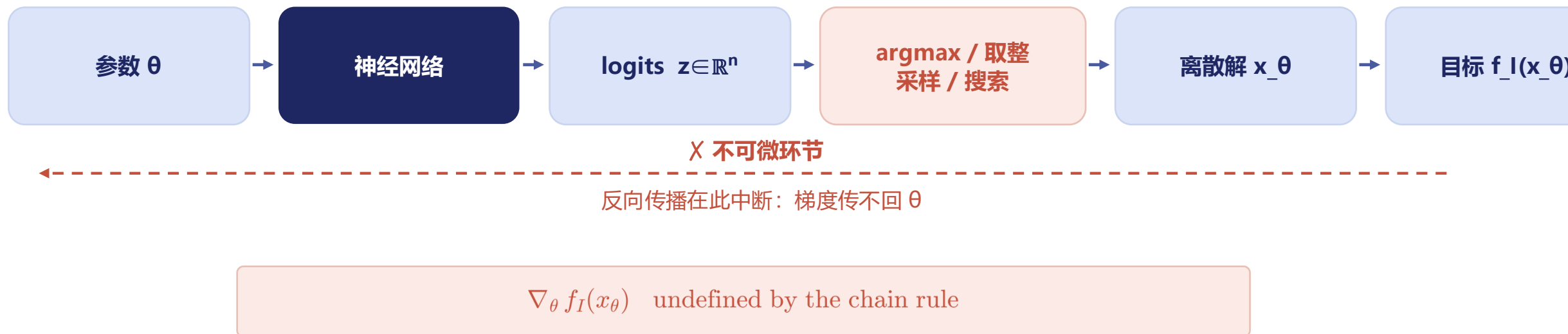
编码的意义 图着色由此变成 **0-1 整数可行性问题**：同一套整数规划 / SAT 工具即可处理；这类“翻译”是离散优化的通用第一步。

困难所在：离散输出切断了梯度

考虑一般离散优化

$$x^* = \arg \min_{x \in \mathcal{X}(I)} f_I(x)$$

——能否让网络直接“梯度下降”出 x^* ？



- 映射 $\theta \mapsto x_\theta$ 含 argmax、取整、采样或组合搜索，**在通常意义下不可微**
- 因此不能像连续优化那样，对离散变量直接做梯度下降

现代解法的共同点 **不是取消离散性**，而是在“构造、松弛、能量、迭代、搜索”五个环节之一引入可学习结构——下页总览。

神经离散优化的五种范式（本章路线图）

范式一 自回归构造

学习逐步选择离散决策的策略，掩码保证可行

Pointer Net / Transformer

范式二 连续松弛

离散变量暂入连续空间，最后取整或投影

$\sigma(z) \cdot \text{Gumbel} \cdot \text{Sinkhorn}$

范式三 能量模型

目标视为能量，学习单调下降的动力学

QUBO / Hopfield

范式四 学习迭代算法

把求解器展开成 T 步可学习迭代

学习型优化器

范式五 增强精确求解器

学习分支、剪枝与排序，正确性交给经典算法

B&B / SAT + GNN

三条主线

直接生成解 — 范式一

连续化后再离散 — 二、三、四

增强传统求解器 — 范式五

统一目标 五种范式都在回答同一问题：在哪个环节注入可学习结构，既保留离散约束，又让梯度有路可走？

概率视角：学习“解的条件分布”

让神经网络定义给定实例下解的条件分布，并在实例分布上优化期望目标：

$$p_{\theta}(x | I)$$

$$\min_{\theta} \mathbb{E}_{I \sim \mathcal{D}} \mathbb{E}_{x \sim p_{\theta}(\cdot | I)} [f_I(x)]$$

- $\mathcal{D}$ ：问题实例的分布（不是单个实例！）
- **经典算法**：针对单个实例从头求解
- **神经求解器**：从大量相关实例中学习可迁移的搜索经验

经典视角

每个实例独立求解：

实例 $I \rightarrow$ 算法 $\rightarrow$ 解 x^*

知识不在实例间共享

学习视角

在分布上摊销求解成本：

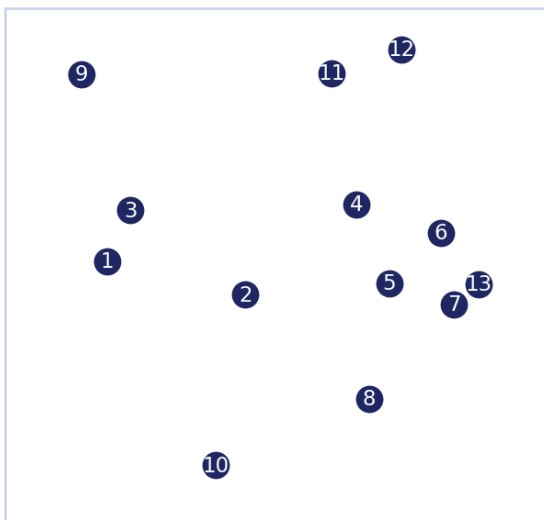
训练分布 $\mathcal{D} \rightarrow$ 策略 $p_{\theta} \rightarrow$ 新实例秒答

重复结构被压进参数 θ

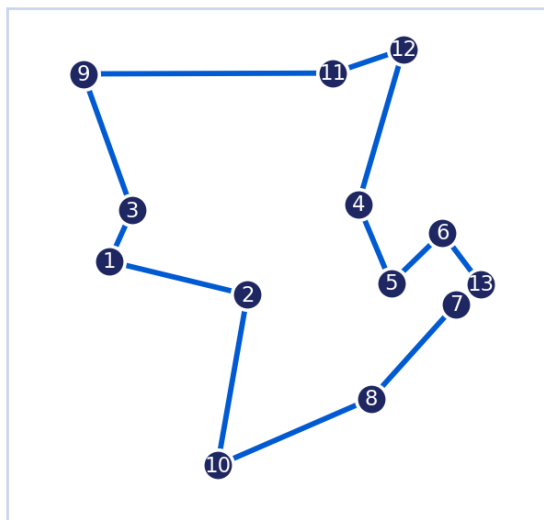
本质（核心观点） 神经网络并没有把 NP-hard 问题自动变成多项式时间问题。它利用训练分布中的**重复结构**，学习更好的候选解生成器、连续代理或搜索启发式，从而降低典型实例的求解时间或提高近似质量。

范式一·旅行商问题 (TSP) 的设置

问题实例： n 个城市



解：一条排列 $\pi \in S_n$



给定平面上的城市：

$$v_1, \dots, v_n \in \mathbb{R}^2$$

解是一条排列：

$$\pi = (\pi_1, \dots, \pi_n) \in S_n$$

最小化闭合路径长度：

$$L(\pi) = \sum_{t=1}^n d(v_{\pi_t}, v_{\pi_{t+1}}), \quad \pi_{n+1} = \pi_1$$

- 解空间规模 $(n-1)!/2$ ： $n=30$ 已是天文数字
- 但“好路径”有几何结构：近邻相连、避免交叉

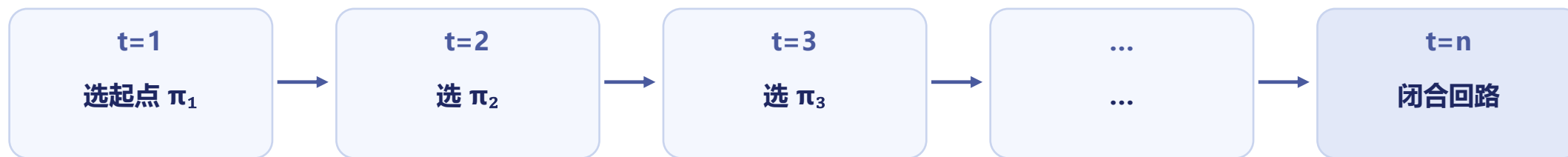
构造式思路 网络不必一次输出整个排列——可以像“走一步看一步”那样，按顺序逐城决策。

范式一 · 自回归分解：逐步构造排列

把整条路径的概率拆成逐步条件概率的乘积：

$$p_{\theta}(\pi | I) = \prod_{t=1}^n p_{\theta}(\pi_t | \pi_1, \dots, \pi_{t-1}, I)$$

解码过程（每步一个离散决策）：



每一步：网络读取（问题实例 I ，当前部分解 $\pi_{1:t}$ ） $\rightarrow$ 输出下一城市的分布

定义（构造型神经求解器）： 学习映射（问题实例, 当前部分解） $\mapsto$ 下一个离散动作；Pointer Network、Transformer 解码器与图神经网络均可参数化这一策略。

链式好处 整体组合爆炸被分解为 n 个“从剩余城市里挑一个”的局部决策。

范式一 · 掩码 softmax：把可行性焊进网络

第 t 步尚未访问的城市集合：

$$\mathcal{A}_t = V \setminus \{\pi_1, \dots, \pi_{t-1}\}$$

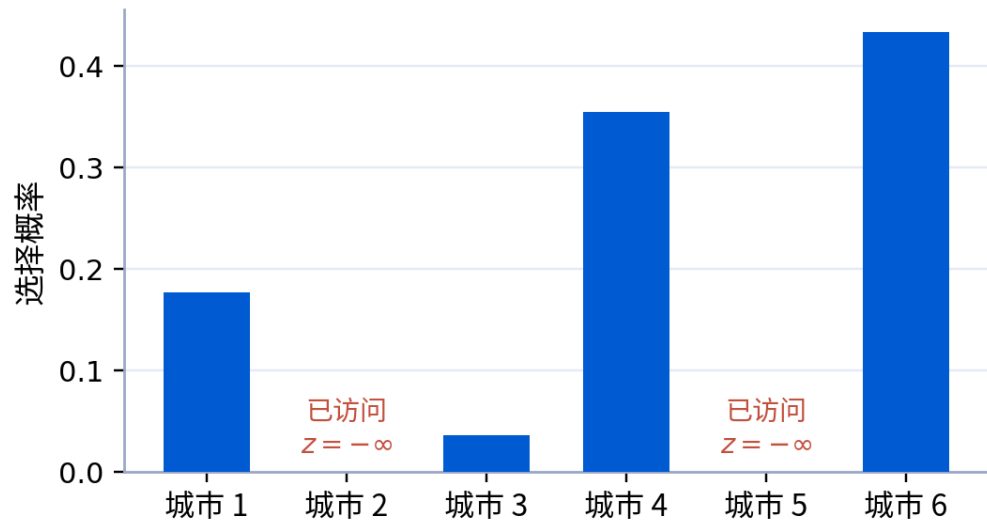
网络输出 logits，仅在可行集合上归一化：

$$p_{\theta}(\pi_t = i \mid \pi_{<t}, I) = \frac{\exp z_{t,i}}{\sum_{j \in \mathcal{A}_t} \exp z_{t,j}}, \quad i \in \mathcal{A}_t$$

对已访问城市直接屏蔽：

$$z_{t,i} = -\infty, \quad i \notin \mathcal{A}_t \implies p = 0$$

掩码 softmax：已访问城市概率恒为 0



- “每个城市只访问一次” 被**结构性保证**
- 无需事后修复，生成的排列天然可行

设计哲学 能用结构保证的约束，绝不留给损失函数——掩码机制把约束直接编码进解码器，是范式一最重要的细节。

范式一·推理：贪心解码与 beam search

贪心： 每步选当前概率最大的城市

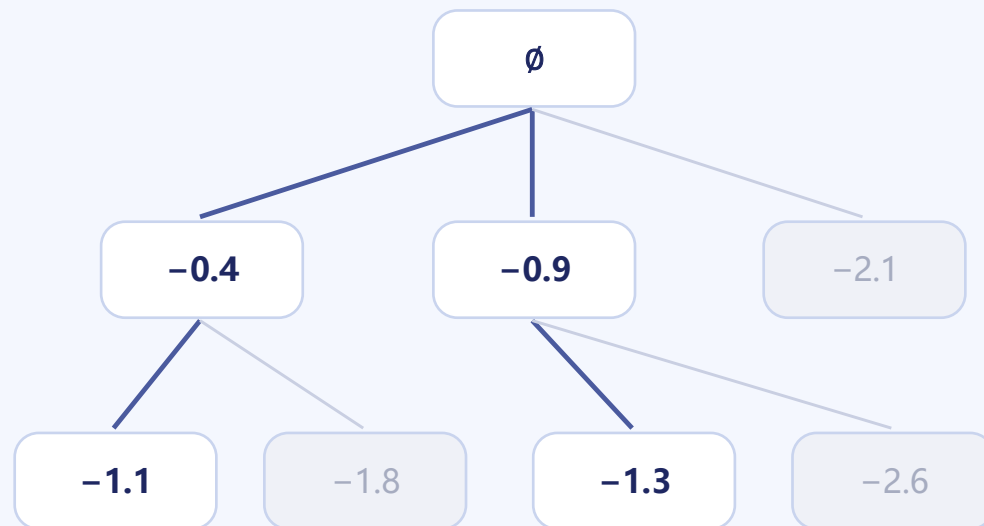
$$\pi_t = \arg \max_{i \in \mathcal{A}_t} p_{\theta}(i \mid \pi_{<t}, I)$$

Beam search： 保留累计对数概率最高的 B 条部分路径

$$\text{score}(\pi_{1:t}) = \sum_{s=1}^t \log p_{\theta}(\pi_s \mid \pi_{<s}, I)$$

- **贪心：** 最快，但一步走错全程偏离
- **beam (宽度 B)：** 每层只留 Top-B，受控地探索组合空间
- **分工：** 网络给“排序函数”，beam 做“显式搜索”

Beam search (B = 2)



每层按累计 log-prob 排序，灰色分支被剪掉

网络负责打分，搜索负责保底

推理谱系 贪心 $\subset$ beam $\subset$ 采样多解取最优：算力换质量，按需选择。

策略梯度：只用目标值就能训练构造策略

没有最优解标签？只要每条路径的长度可以计算，就能直接最小化期望代价：

$$J(\theta) = \mathbb{E}_{\pi \sim p_{\theta}(\cdot | I)} [L(\pi)]$$

关键恒等式（对数导数技巧）：

$$\nabla_{\theta} p_{\theta}(\pi) = p_{\theta}(\pi) \nabla_{\theta} \log p_{\theta}(\pi)$$

代入期望，得 REINFORCE 梯度：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi \sim p_{\theta}} [L(\pi) \nabla_{\theta} \log p_{\theta}(\pi | I)]$$

自回归分解让信号传到每一步：

$$\log p_{\theta}(\pi | I) = \sum_{t=1}^n \log p_{\theta}(\pi_t | \pi_{<t}, I)$$

- 一次完整路径的好坏，为所有局部决策提供学习信号
- **期望对 θ 可微**，尽管采样本身是离散的

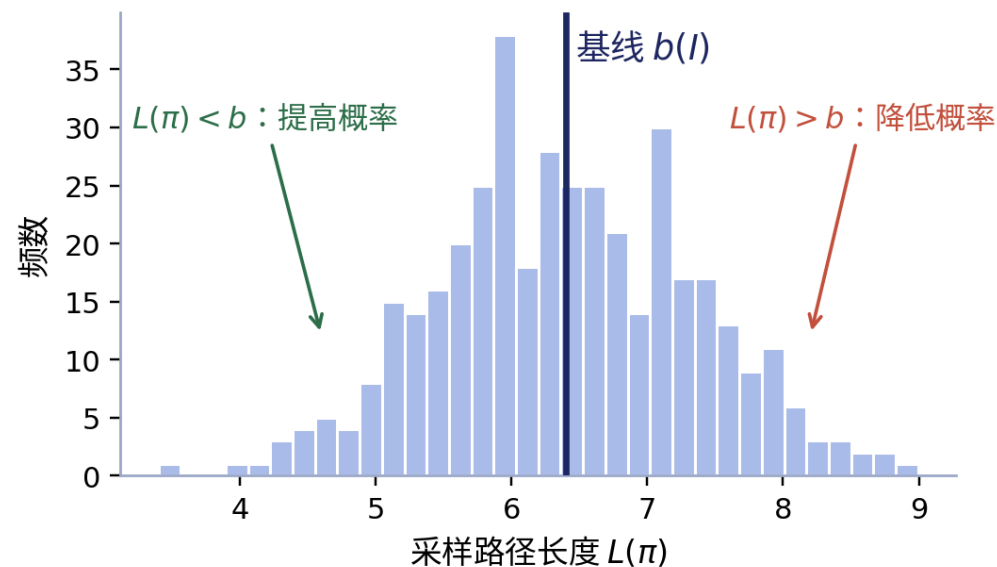
直觉 不要求“知道哪条路最优”，只要求“能给走过的路打分”——这就是强化学习入场的门槛。

基线：降低策略梯度的方差

引入只依赖实例、不依赖当前采样动作的基线 $b(I)$ ，梯度无偏且方差更小：

$$\nabla_{\theta} J(\theta) = \mathbb{E} [(L(\pi) - b(I)) \nabla_{\theta} \log p_{\theta}(\pi | I)]$$

- $L(\pi) < b(I)$ ：优于基线 $\Rightarrow$ 提高该路径的概率
- $L(\pi) > b(I)$ ：差于基线 $\Rightarrow$ 降低相应动作概率
- **常用基线**：同实例多次采样的均值、贪心 rollout、学习的价值网络



以基线为参照：比平均更短 $\rightarrow$ 强化；更长 $\rightarrow$ 抑制

提示（强化学习的适用条件）：不要求预知最优解，只要求能对完整或部分解计算奖励；代价是梯度方差较大、信用分配困难，且通常需要大量训练实例与采样。

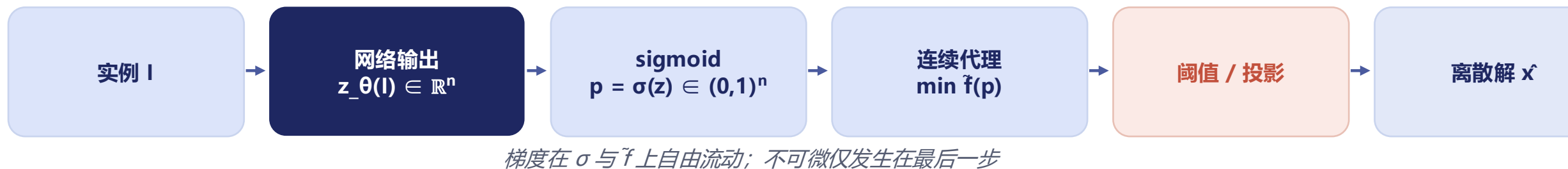
小结（范式一） 自回归构造 + 掩码可行性 + 策略梯度训练 = 一台可以自我改进的“构造机”。

范式二 · 连续松弛：绕过不可微的取整

二元优化

$$\min_{x \in \{0,1\}^n} f(x)$$

：让网络先在连续世界里优化，最后再回到离散点。



$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

离散化的两种方式

threshold: $\hat{x}_i = 1$ if $p_i \geq \frac{1}{2}$, else 0

projection: $\hat{x} = \text{Proj}_{\mathcal{X}}(p)$ (constraint-aware)

- 训练全程在连续世界进行；离散化只在推理出口执行一次

代价预告 绕过不可微不是免费的——松弛会引入两种系统性误差，下页给出精确分解。

范式二 · 两种误差：松弛间隙与取整误差

连续可行域 $\tilde{\mathcal{X}} \supseteq \mathcal{X}$ ，连续最优解：

$$p^* = \arg \min_{p \in \tilde{\mathcal{X}}} \tilde{f}(p)$$

最终离散解的损失可精确拆成两项：

$$f(\hat{x}) - f(x^*) = [f(\hat{x}) - \tilde{f}(p^*)] + [\tilde{f}(p^*) - f(x^*)]$$

第一项

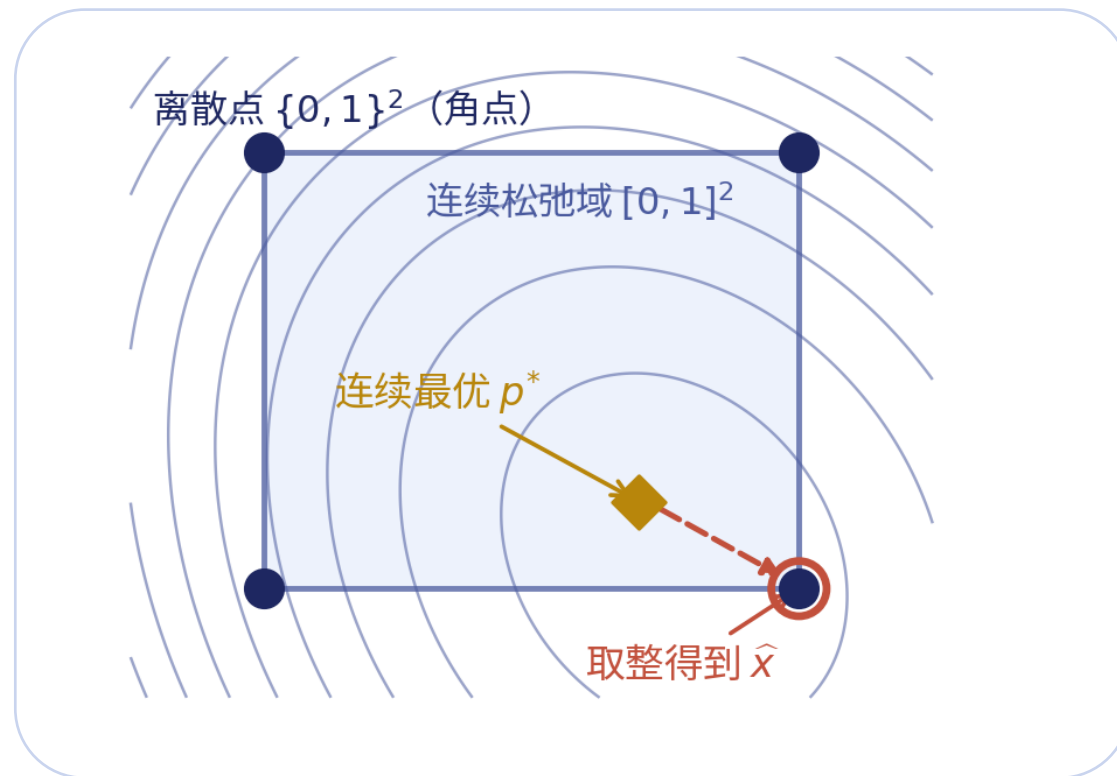
取整 / 投影误差

从 p^* 回到离散点的损失

第二项

松弛间隙

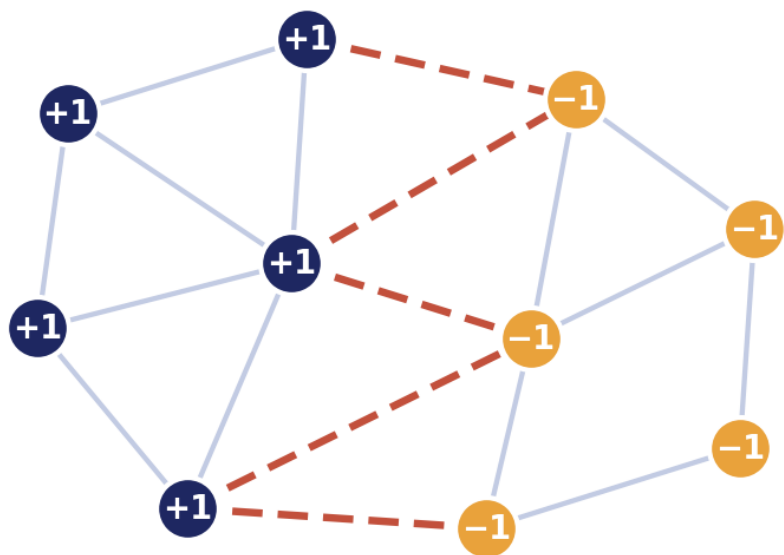
连续最优本就低于离散最优



连续最优 p^* (◆) 落在域内部，取整后跳到角点 $\hat{x}$ (○)

工程含义 代理设计要让两项都小：松弛要“紧”（贴近离散包络），取整要“稳”（ p 接近 0/1 再离散化）。

示例：Max-Cut 的离散形式



被切断的边（红色虚线）：5 条

每个节点取一侧： $x_i \in \{-1, +1\}$

边 (i, j) 被切断的指示量：

$$\frac{1 - x_i x_j}{2} \in \{0, 1\} \quad (= 1 \Leftrightarrow x_i \neq x_j)$$

于是 Max-Cut 写成自旋形式：

$$\max_{x_i \in \{-1, +1\}} \frac{1}{2} \sum_{(i, j) \in E} w_{ij} (1 - x_i x_j)$$

NP-hard；但目标是 x 的光滑多项式——天然适合连续松弛。

松弛入口 把 $x_i \in \{-1, +1\}$ 放松为 $s_i \in [-1, 1]$ ，同一个目标立即变成可微函数——这正是 GNN 的用武之地（下页）。

GNN 求解 Max-Cut: 学习一个图上迭代算法

GNN 为每个节点输出连续值 $s_i \in [-1,1]$, 最小化:

$$\widetilde{L}(s) = -\frac{1}{2} \sum_{(i,j) \in E} w_{ij}(1 - s_i s_j)$$

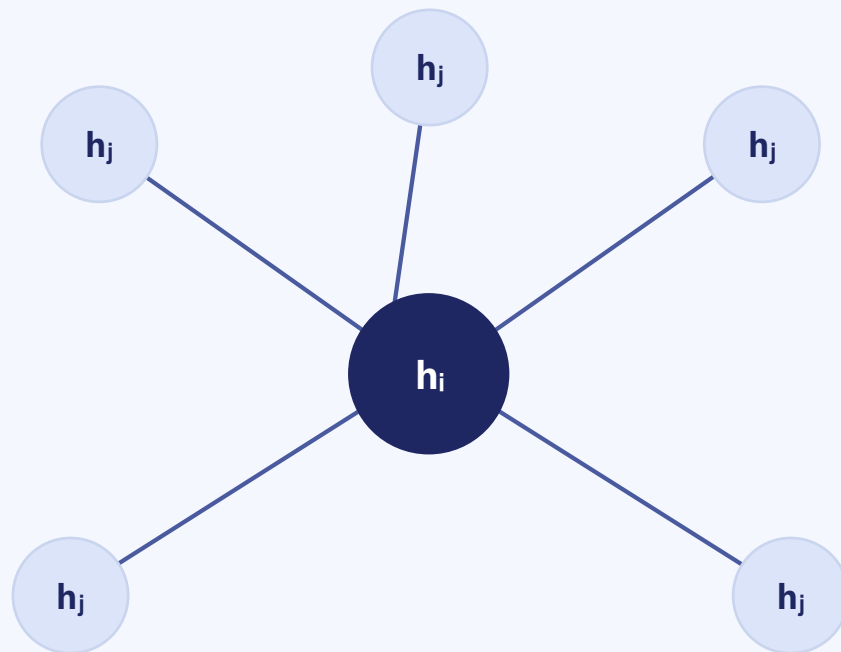
节点表示按图结构迭代更新 (消息传递):

$$h_i^{(\ell+1)} = \phi_\theta \left(h_i^{(\ell)}, \sum_{j \in \mathcal{N}(i)} \psi_\theta(h_i^{(\ell)}, h_j^{(\ell)}, w_{ij}) \right)$$

读出与离散化:

$$s_i = \tanh(Wh_i^{(L)}), \quad \widehat{x}_i = \text{sign}(s_i)$$

消息传递: 邻居信息汇聚到中心



每层 = 一次局部更新; L 层 = 一个 L 步迭代算法

换个角度看 GNN 相当于学习一个局部消息传递型的图上迭代优化算法——架构本身就是“算法的形状”。

示例：图着色的软分配与熵正则

让网络输出软分配矩阵（每行是颜色分布）：

$$P = (p_{ic}) \in [0, 1]^{|V| \times K}, \quad \sum_{c=1}^K p_{ic} = 1$$

软冲突损失（相邻同色的“概率质量”）：

$$L_{\text{conflict}} = \sum_{(i,j) \in E} \sum_{c=1}^K p_{ic} p_{jc}$$

熵项促使每行趋向 one-hot：

$$L(P) = L_{\text{conflict}} + \lambda H(P), \quad \hat{c}_i = \arg \max_c p_{ic}$$

对严格约束：优先使用掩码、可行性保持构造、投影或后处理修复。

软分配 $P \rightarrow \arg\max$ 离散化

	红	蓝	绿
v1	0.86	0.09	0.05
v2	0.12	0.71	0.17
v3	0.48	0.45	0.07
v4	0.05	0.11	0.84

← v₃ 仍摇摆：
argmax 可能出错

警示：软约束 ≠ 硬可行性

惩罚项只能“鼓励”接近可行域；argmax 后仍可能相邻同色，需贪心重着色、局部搜索或精确求解器修复。

对照范式一 掩码 = 结构性保证可行；软惩罚 = 概率上倾向可行。能选前者就别赌后者。

可微离散采样: Gumbel-Softmax

要从 K 类中选 one-hot: $x \in \{e_1, \dots, e_K\}$

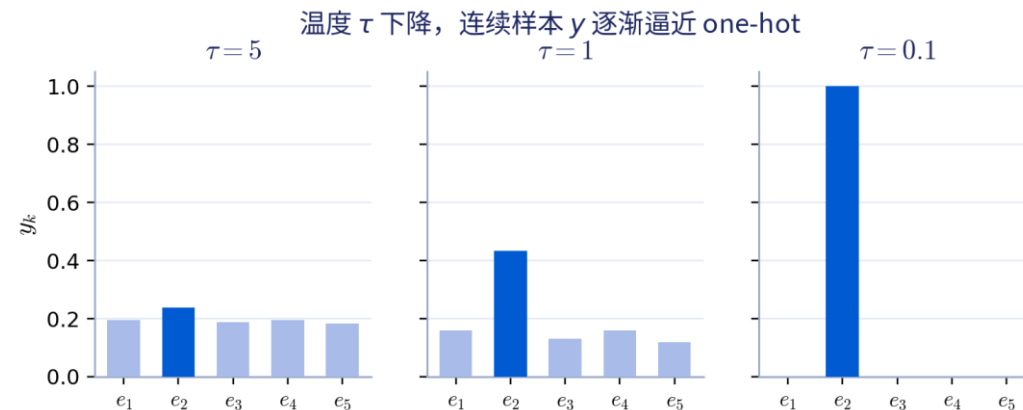
加 Gumbel 噪声 $\mathbf{g}_k \sim \mathrm{Gumbel}(0, 1)$, 做带温度的 softmax:

$$y_k = \frac{\exp((z_k + g_k)/\tau)}{\sum_{j=1}^K \exp((z_j + g_j)/\tau)}$$

温度趋零时收敛到离散选择:

$$\tau \rightarrow 0 \implies y \rightarrow e_{k^*}, \quad k^* = \arg \max_k (z_k + g_k)$$

- **训练:** 对连续 y 反向传播 (梯度存在)
- **推理:** 用离散 $\arg \max$ (决策干脆)



为什么要加噪声?

Gumbel 噪声让 $\arg \max(\mathbf{z} + \mathbf{g})$ 的分布恰好等于按 $\text{softmax}(\mathbf{z})$ 采样——既保留随机探索, 又给出可微的连续近似。

定位 这是“连续松弛”家族的随机版本: 松的是采样过程本身。

排列的松弛：Sinkhorn 与双随机矩阵

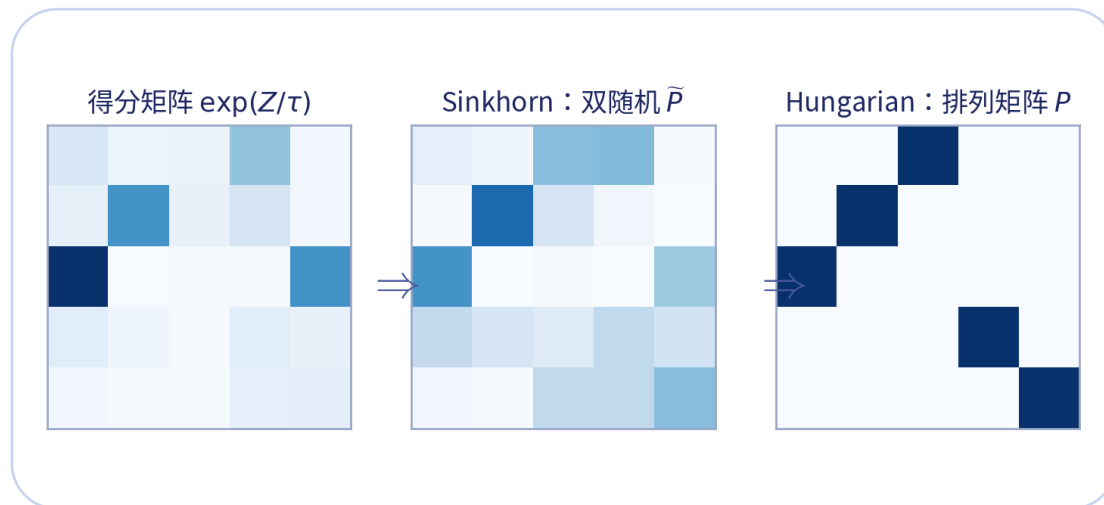
排列矩阵 = 0/1 矩阵且每行每列恰好一个 1:

$$P \in \{0, 1\}^{n \times n}, \quad P\mathbf{1} = \mathbf{1}, \quad P^\top \mathbf{1} = \mathbf{1}$$

其凸包是**双随机矩阵**集合。给定得分矩阵 Z ，交替做行归一化与列归一化（Sinkhorn 迭代）：

$$\tilde{P} = \text{Sinkhorn}(Z/\tau): \quad \tilde{P}_{ij} \geq 0, \quad \tilde{P}\mathbf{1} = \mathbf{1}, \quad \tilde{P}^\top \mathbf{1} = \mathbf{1}$$

- **训练**：在双随机的 $\tilde{P}$ 上反向传播
- **推理**：用 Hungarian 算法把 $\tilde{P}$ 投影成真排列
- 与 Gumbel 同理：连续近似训练，离散投影输出



提示：温度退火的作用

较大的 τ 提供平滑梯度但输出远离离散点；较小的 τ 接近 one-hot / 排列但梯度尖锐不稳。常在训练中逐步降低 τ 。

应用面 指派、排序、匹配、TSP 解的重排——一切“排列变量”问题，都可经 **Sinkhorn 松弛 + Hungarian 投影** 进入可微世界。

范式三 · 能量模型：QUBO 与 Hopfield 网络

许多二元组合优化可写成二次无约束二元优化（QUBO）：

$$\min_{x \in \{0,1\}^n} E(x), \quad E(x) = x^\top Qx + c^\top x$$

换成自旋变量 $s_i \in \{-1, +1\}$ ，得 Hopfield 能量：

$$E(s) = -\frac{1}{2} \sum_{i,j} W_{ij} s_i s_j - \sum_i b_i s_i$$

异步更新一个坐标：

$$s_i \leftarrow \text{sign}\left(\sum_j W_{ij} s_j + b_i\right)$$

关键性质：能量单调下降

当 W 对称、对角线适当处理时：

$$E(s^{(t+1)}) \leq E(s^{(t)})$$

因此网络状态终将落入某个局部极小点

为什么这算“求解”？

把组合问题的目标写成能量 E ，

“运行网络” = “最小化目标”

优化嵌入在动力学里，而非外部算法。

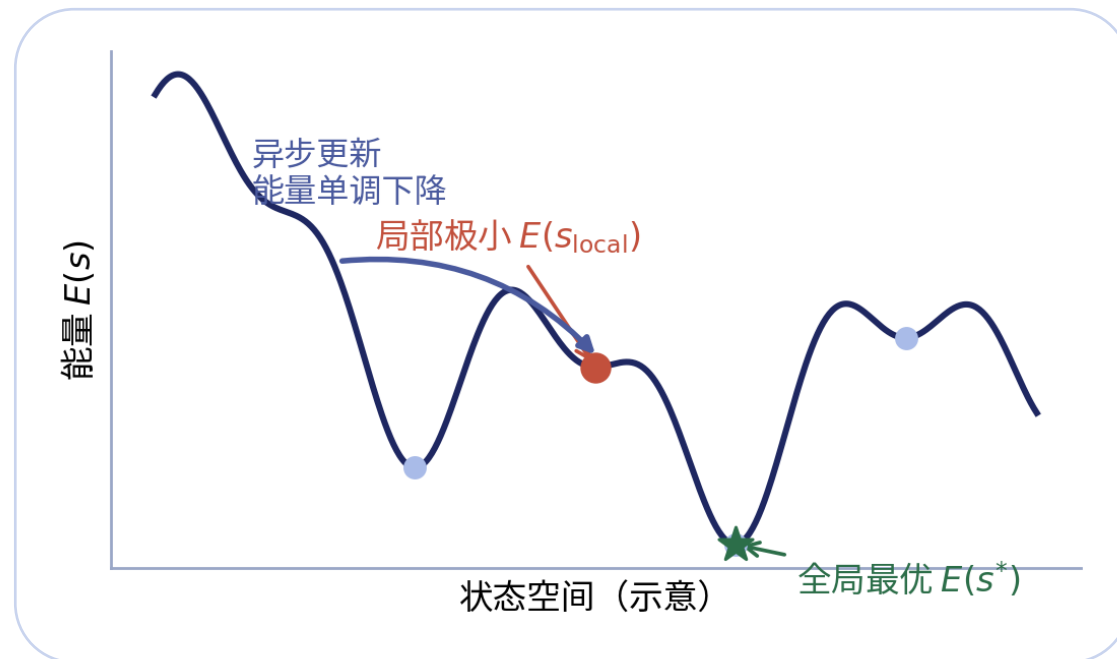
一句话 能量模型把求解器做成一个物理系统：状态自发滚向能量低谷，谷底即（局部）最优解。

范式三 · 学习动力学，与局部极小的现实

把求解器写成可学习的迭代映射，并要求能量类似 Lyapunov 函数：

$$s^{(t+1)} = \Phi_{\theta}(s^{(t)}, I), \quad E_I(s^{(t+1)}) \leq E_I(s^{(t)})$$

- 经典 Hopfield：更新规则由固定能量决定
- 现代方法：学习更新算子 Φ_{θ} ，自适应步长与方向
- 跳出差的局部极小：随机扰动、退火、多起点、局部搜索



警示：能量下降不保证全局最优

单调下降只排除“越走越差”，不排除收敛到次优局部极小：

若需要全局最优性，仍需分支定界、可验证下界或其他精确算法。

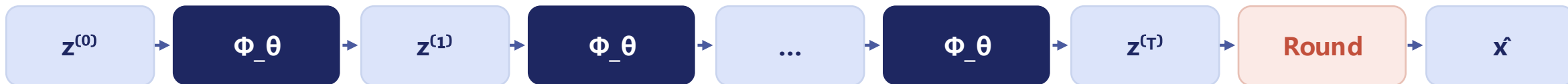
$$E(s_{\text{local}}) > E(s^*)$$

诚实的承诺 能量法承诺“不会变差”，不承诺“一定最好”——这正是范式五存在的理由。

范式四 · 把“优化算法本身”交给网络学习

网络不直接输出解，而是展开为 T 步迭代——每步由可学习算子驱动：

$$z^{(t+1)} = \Phi_{\theta}(z^{(t)}, I, \nabla \tilde{f}_I(z^{(t)})), \quad p^{(t)} = \sigma(z^{(t)}), \quad \hat{x} = \text{Round}(p^{(T)})$$



同参数每步复用 $\approx$ 循环网络；逐步不同参数 $\approx$ 展开网络

对比：传统梯度法（固定规则）

$$z^{(t+1)} = z^{(t)} - \eta_t \nabla \tilde{f}_I(z^{(t)})$$

步长与方向写死，对所有实例一视同仁

学习型优化器（自适应规则）

Φ_{θ} 可依据当前残差、局部图结构、历史轨迹与约束违反程度，自适应产生更新；训练目标可只看终态，也可监督整条轨迹。

定义（学习型优化器） 学的不是某个实例的答案，而是“(当前状态, 问题结构) $\mapsto$ 下一步搜索方向”的更新规律。

训练信号的三种来源

监督学习

模仿已知最优解

$$L_{\text{sup}}(\theta) = - \sum_{k=1}^N \log p_{\theta}(x_k^* | I_k)$$

- 稳定、易训练
- 需精确求解器造标签
- 多个等价最优解时，单标签会误罚其他最优解

强化学习

直接优化目标值

$$\max_{\theta} \mathbb{E}_I \mathbb{E}_{x \sim p_{\theta}}[R_I(x)], \quad R_I = -f_I$$

- 无需最优解标签
- 高方差、稀疏奖励
- 需要大量采样

无监督 / 自监督

目标函数即损失

$$L_{\text{unsup}} = \tilde{f}_I(p_{\theta}) + \lambda C_I(p_{\theta})$$

- 不需标签或策略梯度
- 依赖松弛与取整质量
- 约束违反项： $[\mathbf{g}]_+^2 + \mathbf{h}^2$

例 (如何选)： 有大量精确标签 → 监督；只能算目标值 → 强化；有良好可微松弛 → 无监督。实际系统常组合使用：先监督预训练，再用强化学习或目标函数微调。

范式五 · 神经网络增强精确求解器 (MIP)

更可靠的路线：网络只学习影响效率、不影响正确性的启发式决策。以混合整数规划为例：

$$\min c^\top x, \quad Ax \leq b, \quad x_i \in \mathbb{Z}$$

分数变量 $x_i^{\text{LP}} \notin \mathbb{Z}$ 触发分支：

$$x_i \leq \lfloor x_i^{\text{LP}} \rfloor \quad \text{or} \quad x_i \geq \lceil x_i^{\text{LP}} \rceil$$

网络为候选分支变量打分并择优：

$$s_i = f_\theta(\text{node}, x_i), \quad i^* = \arg \max_i s_i$$

网络可以学习哪些决策？

- 选择哪个分数变量进行分支
- 决定优先搜索哪个分支节点
- 对候选切割平面进行排序
- 预测一个高质量初始可行解
- 选择局部搜索的（大）邻域变量集

关键： 这些选择只改变搜索树形状，**不破坏分支定界的完备性**；选差了顶多变慢，最优解与证书仍然可得。

效率与正确性的分工（核心观点） 神经网络负责候选与排序 + 精确求解器负责约束、下界和证书——与定理证明中“模型生成候选、内核检查证明”是同一个生成器-验证器结构。

范式五 · SAT 求解中的变量选择

CNF 公式: $F = C_1 \wedge C_2 \wedge \dots \wedge C_m$

DPLL / CDCL 求解器需不断选择变量及其赋值方向; 最坏情况要搜索:

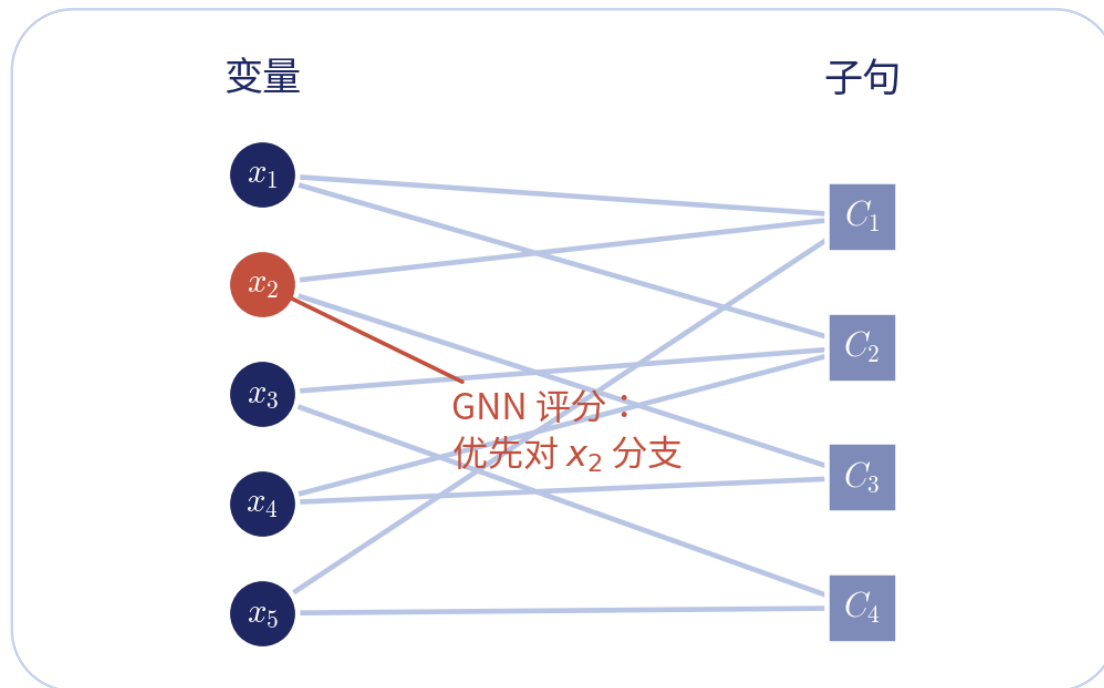
$O(2^n)$ assignments

但优秀的变量选择策略可以做到:

$$N_{\text{visited}} \ll 2^n$$

把公式表示为变量-子句二部图, GNN 估计:

$$p_\theta(x_i \text{ should branch first} \mid G)$$



二部图 $G = (V_{\text{var}} \cup V_{\text{clause}}, E)$: 边表示“变量出现在该子句中”——结构信息正是好分支的线索。

定位 一个典型的“机器学习增强离散求解器”: 分支顺序学出来, 正确性仍由 CDCL 的逻辑推理保证。

如何评价一个神经离散优化方法？

1 可行性

$$\hat{x} \in \mathcal{X}(I)$$

掩码 / 可行性保持构造可结构性保证；软罚函数只能让违反量 ≈ 0 ，不能保证严格为零。

2 解质量：最优性间隙

$$\text{gap} = \frac{U_I - L_I}{\max\{1, |U_I|\}}$$

纯神经法通常只给上界 U ；与分支定界结合后才可能同时获得下界与证书。

3 分布外泛化

$$\mathcal{D}_{\text{test}} \neq \mathcal{D}_{\text{train}}$$

规模、度分布、约束密度变化都可能掉点： $n=50$ 训练 $\neq n=500$ 可用。

4 复杂度含义

$$T_{\text{infer}}(n) \ll T_{\text{exact}}(n)$$

即使典型实例更快，也推不出最坏情况复杂度被降低；训练成本被分摊到同分布实例上。

警示：经验速度 $\neq$ 理论复杂度

在基准上更快，通常是平均实例的常数、搜索树规模或近似质量改善；除非给出严格最坏情况分析，不能声称得到新的多项式时间算法。

神经离散优化的统一数学框架

五种范式可统一为：学习条件分布 / 策略，并最小化期望目标 + 约束违反：

$$\min_{\theta} \mathbb{E}_{I \sim \mathcal{D}} \mathbb{E}_{x \sim p_{\theta}(\cdot | I)} [f_I(x) + \lambda C_I(x)]$$

方法

解的产生机制

自回归构造

$$p_{\theta}(x | I) = \prod_t p_{\theta}(x_t | x_{<t}, I)$$

连续松弛

$$p = \sigma(z_{\theta}(I)), \quad x = \text{Round}(p)$$

能量模型

$$p_{\theta}(x | I) \propto \exp[-E_{\theta}(x, I)]$$

学习型迭代

$$z^{(t+1)} = \Phi_{\theta}(z^{(t)}, I)$$

神经引导搜索

$$a_t \sim \pi_{\theta}(a_t | s_t)$$

方法选择原则 要推理快、允许近似 → 直接生成 / 连续松弛；约束必须严格满足 → 可行性保持解码或修复；**必须有全局最优与证书** → 让神经网络增强而非替代精确求解器。

PART 5

复杂度优化、自动发现与统一视角

程序搜索、复杂度的四个层次，以及一个理想的研究闭环

- 自动算法发现与程序搜索
- 复杂度优化的四个层次（常数 / 平均 / 参数化 / 渐近）
- 离散数学中的自动反例搜索
- 统一视角：三个方向都是约束搜索

自动算法发现与程序搜索

在候选程序空间 $\mathcal{P}$ 中寻找实现目标函数的程序：

$$P^* = \arg \min_{P \in \mathcal{P}} \left[\sum_{x \in \mathcal{D}} 1 \{P(x) \neq f(x)\} + \lambda \text{Cost}(P) \right]$$

代价 $\text{Cost}(P)$ 可以衡量：

运算次数

乘法次数

内存访问量

并行深度

通信复杂度

但：有限测试集上的正确性——

$$P(x_i) = f(x_i), \quad i = 1, \dots, N \quad / \Rightarrow \quad \forall x \in \mathcal{X}: P(x) = f(x)$$

快但只对了 99.99% 的算法，对数学而言等于错。

生成器-验证器，再次登场

AI 搜索

候选程序 P

形式验证

程序等价性证明

不等价 → 反例返回，继续搜索

机器可以大胆探索奇形怪状的程序，
最终结果仍具备可验证的正确性保证。

例子方向 矩阵乘法方案、排序网络、位运算技巧——“搜索 + 证明等价”已产出真实的新算法。

复杂度优化的四个层次

L1 常数因子

$$100n^2 \rightarrow 2n^2$$

L2 平均复杂度

$$\mathbb{E}_{I \sim \mathcal{D}}[T(I)] \downarrow$$

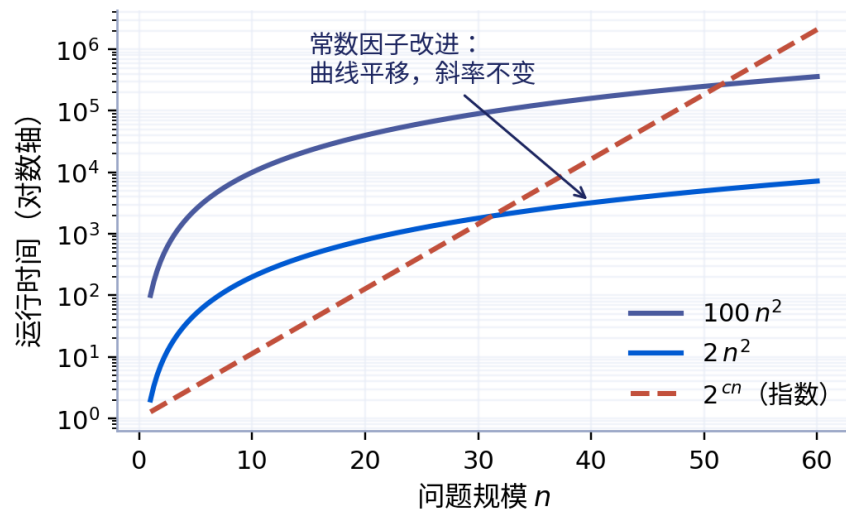
L3 参数化复杂度

$$T(n, k) = f(k) \text{poly}(n)$$

L4 渐近复杂度

$$O(n^\alpha) \rightarrow O(n^\beta), \beta < \alpha$$

阶梯关系：AI 方法多在 L1-L2 发力；L3 需要分析分支结构；L4 是复杂度理论的圣杯。



逐行解读：

L1 = 曲线整体下移（斜率不变）

L2 = “典型输入” 那部分曲线下移

L4 = 斜率本身变小

警示：工程更快、GPU 并行度更高、常数更小，都不意味着渐近复杂度改变。讨论“算法改进”必须说清在哪一层。

离散数学中的自动反例搜索

把“找反例”改写成存在性搜索问题：

$$\forall G \in \mathcal{G}: P(G) \quad \stackrel{?}{\iff} \quad \nexists G \in \mathcal{G}: \neg P(G)$$

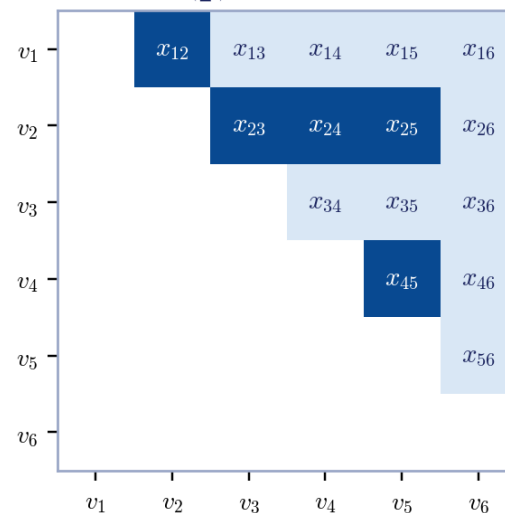
固定顶点数 n ，一个简单无向图由 $\mathbf{C(n, 2)}$ 个布尔变量完全编码：

$$x_{ij} = 1 \iff (i, j) \in E, \quad x_{ij} = 0 \iff (i, j) \notin E$$

再把图的结构条件与猜想的否定 $\neg P$ 一并编码为 SAT、SMT 或整数规划，交给求解器。

SAT → 得到一个明确反例图； **UNSAT + 证书** → 该规模下不存在反例（可独立检查）。

一个图 = $\binom{n}{2}$ 个布尔变量（上三角）



UNSAT 的含义（对固定 n ）：

$$\forall G: |V(G)| = n \implies P(G)$$

价值 反例搜索是猜想的“压力测试”：撑过 SAT 求解器的轰炸，再去找证明也不迟。

统一视角：三个方向都是约束搜索

定理证明

候选空间 = 证明序列 Π

$$\pi^* \in \Pi: \text{KernelCheck}(\pi^*, \varphi) = \text{True}$$

规律发现

候选空间 = 公式类 $\mathcal{H}$

$$f^* = \arg \min_{f \in \mathcal{H}} [\text{FitErr}(f) + \lambda \text{Cmplx}(f)]$$

算法优化

候选空间 = 程序 / 策略 $\mathcal{P}$

$$P^* = \arg \min_P \text{Cost}(P) \text{ s.t. } \text{Correct}(P)$$

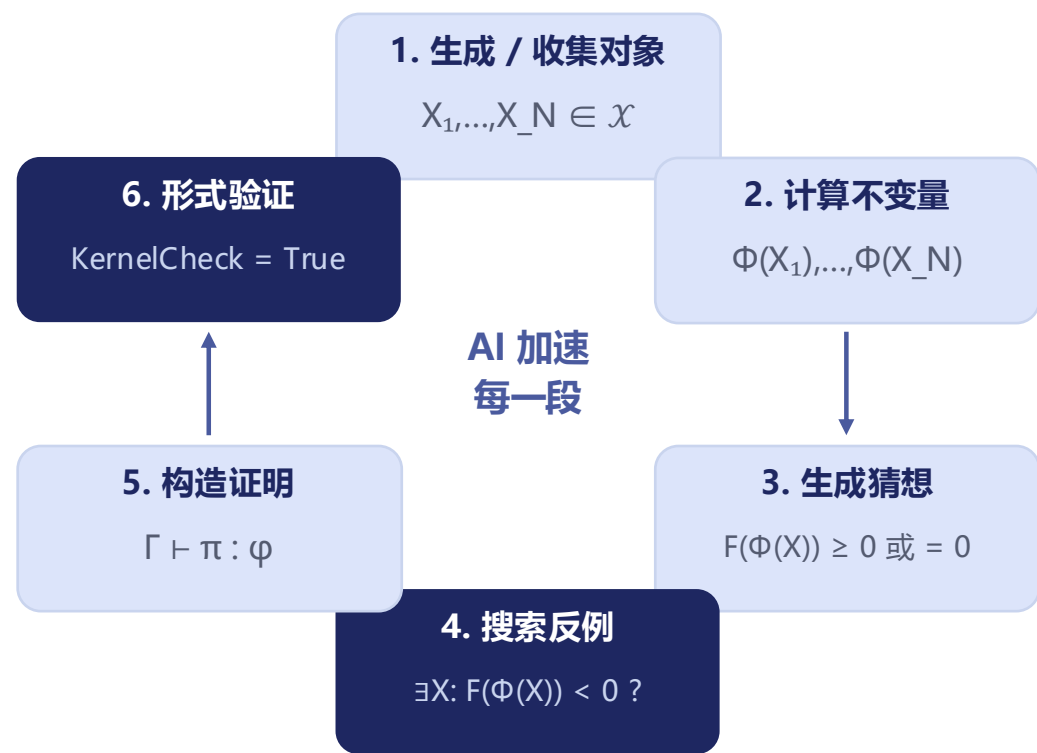


$$\text{find } s \in \mathcal{S}: \mathcal{C}(s) = \text{True}, \min J(s)$$

同一个模板：约束搜索

工程含义 同一套“生成器 + 验证器 + 搜索器”基础设施，可以在三个方向之间复用与迁移。

理想的数学研究闭环，与数学家的位置



数学家的不可替代作用（核心观点）

- 哪些定义真正抓住了问题本质
- 哪些不变量值得研究
- 哪个中间引理具有结构性意义
- 一个证明是否给出了**概念解释**，而不仅仅是形式正确

AI 高效完成大规模枚举、局部证明补全、候选公式生成与反例搜索。

协作图景 机器跑圈，人类掌舵：循环越快，对“选题与解释”这一人类输入的杠杆就越大。

总结：迈向可信的 AI 数学系统

本讲四条线索，对应数学研究的不同阶段：



一个可信的 AI 数学系统应同时具备：



最终标准 不只是给出一个看似合理的答案，而是提供可解释的结构、可重复的计算过程、能被独立检查的证明证书。